

```

import pandas as pd
import numpy as np

from sklearn.datasets import fetch_california_housing
from scipy.stats import ks_2samp

# 1. Setup Data
data = fetch_california_housing(as_frame=True)
df = data.frame

# Split into "Past" (Training) and "Present" (Production)
training_data = df.sample(frac=0.5, random_state=42)
production_data = df.drop(training_data.index)

# 2. Simulate Production Drift
# Scenario: A massive real estate bubble where prices
# (MedHouseVal)

# and incomes (MedInc) decouple from historical norms.
production_data_drifted = production_data.copy()
production_data_drifted['MedInc'] = production_data['MedInc']
* 0.5 # Income drops
production_data_drifted['AveRooms'] = production_
data['AveRooms'] * 1.8 # Weird data error

# 3. Detection Function
def detect_drift(baseline, current, threshold=0.05):
    report = []
    for col in baseline.columns:
        stat, p_value = ks_2samp(baseline[col], current[col])
        is_drifted = p_value < threshold
        report.append({
            "Feature": col,
            "K-S Stat": round(stat, 4),
            "P-Value": f"{p_value:.2e}",
            "Status": "DRIFTED" if is_drifted else "OK"
        })

    return pd.DataFrame(report)

drift_report = detect_drift(training_data, production_data_
drifted)
print(drift_report)

```

In this code, we use the `ks_2samp` function, which implements the Kolmogorov–Smirnov (K-S) Test. If the P-value is very low (less than 0.05), it indicates that the probability of these two datasets coming from the same distribution is nearly zero. This is your cue to trigger an alert.

In real production systems, training and inference data rarely align perfectly. Features may be missing, new categories may appear, or schemas may evolve. Before

running any drift test, production data must be schema-aligned with training data, any missing values must be handled and categorical levels normalised. Failing to do this can create false drift signals that are just pipeline bugs.

Best practices

Monitoring is a cost centre unless it leads to better outcomes. Here is how to manage it effectively.

Threshold selection and alert fatigue: The most common mistake is setting thresholds too tightly. If every minor fluctuation triggers a Slack notification, your team will eventually ignore the alerts. Only alert on features that have high ‘feature importance’. If a feature that the model barely uses drifts, it might not be worth a midnight page.

When to trigger retraining: Retraining is expensive and risky.

- **Analyse the drift:** Is it a temporary anomaly?
- **Champion-Challenger:** Before replacing the live model, train a ‘Challenger’ model on the drifted data and compare its performance against the ‘Champion’ (live) model using a back test.
- **Data integrity check:** Sometimes ‘drift’ is a bug in the data pipeline. Retraining on ‘bad’ data will only make the model worse.

Human-in-the-Loop: Always include a ‘sanity check’ step. A human data scientist should review a drift report before a model is promoted to production. Monitoring tools should provide ‘explainability’ (SHAP or LIME values) to show why the model is behaving differently.

The monitoring checklist

Building an ML model is a sprint; maintaining it is a marathon. To ensure your production environment remains robust, use this checklist for every new deployment.

- **Establish a baseline:** Save the statistical profiles (mean, variance, K-S stats) of your training data.
- **Feature tiering:** Identify which features are ‘mission critical’ and monitor them with higher sensitivity.
- **Automatic logging:** Ensure your inference pipeline logs every prediction and its input features to a centralised store (like S3 or BigQuery).
- **Rollback plan:** Always have a ‘golden model’ (a simple, robust version) ready to take over if the main model drifts beyond repair.

Start small. You don’t need a complex observability suite on Day 1. Implement a simple K-S test on your top three features, run it once a day, and grow your monitoring strategy as your model’s impact grows. **END** 🐧

By: Aanchal Narendran

The author is a developer in the AI Centre of Excellence at IBM.